

# Early Fault Detection and Risk-Based Maintenance Prioritization for Wind Turbines

Huseynli, Emin

*AI Academy*

*National Artificial Intelligence Center*  
Baku, Azerbaijan

Muradov, Ziyad

*AI Academy*

*National Artificial Intelligence Center*  
Baku, Azerbaijan

Soltanaliyev, Kamal

*AI Academy*

*National Artificial Intelligence Center*  
Baku, Azerbaijan

Yusifli, Ismayil

*AI Academy*

*National Artificial Intelligence Center*  
Baku, Azerbaijan

**Abstract**—Reactive wind turbine maintenance, triggered only after a fault is formally logged, forgoes the opportunity for early intervention. This work investigates whether a recurrent neural network trained on SCADA sensor windows can detect an approaching fault early enough, and with sufficient calibrated confidence, to support risk-based maintenance prioritization across a multi-farm fleet, and whether such a model generalizes to turbines and farms it was not trained on. We frame the task as per-timestep risk regression using a Gated Recurrent Unit over a farm-agnostic physical feature representation, evaluate it against classical baselines and across multiple farms, and report an honest, evidenced account of where the approach succeeds and where domain shift between farms limits it.

**GitHub repository:** <https://github.com/thedarkstringg/team-Sigmoid> (tag `v1.0-final`).

**Index Terms**—wind turbine, fault detection, predictive maintenance, recurrent neural network, domain generalization, SCADA

## I. INTRODUCTION

Wind turbine downtime is costly, and maintenance triggered only after a fault has been formally logged forgoes the opportunity for early, lower-cost intervention. SCADA (Supervisory Control and Data Acquisition) systems already record dense, high-frequency sensor data from operating turbines, raising the question of whether that data alone is sufficient to detect an approaching fault before it is logged, with enough lead time and confidence to act on.

This work investigates that question directly, with two specific research aims. First, we ask whether a recurrent neural network trained on SCADA sensor windows can produce a calibrated, per-timestep fault-risk signal that gives a meaningful lead time ahead of a logged fault, when evaluated against an appropriate classical baseline. Second, and central to this paper, we ask whether such a model *generalizes*: does a model trained on one wind farm’s turbines transfer to a different farm’s turbines, with different hardware, different sensor instrumentation, and different fault-labeling procedures, without any target-domain retraining?

We frame early fault detection as a per-timestep risk regression problem over a 24-hour SCADA window, using a Gated Recurrent Unit (GRU) trained on a farm-agnostic, physically

meaningful feature representation (Section III) so that the same trained model can, in principle, be evaluated on a farm it never saw during training. We evaluate this model against classical baselines and, separately, in a strict zero-shot cross-farm setting (Section V, Section VI), using the CARE-to-Compare multi-farm SCADA dataset [1], which itself provides a published benchmark for comparison.

Rather than reporting only a favorable in-farm result, this paper places equal emphasis on an honest account of where the approach’s generalization breaks down and why. We find that validation performance within a single farm is substantially stronger than held-out test performance, a pattern that persists across six independently tested model configurations and is not resolved by further tuning; we then show, through direct measurement, that this is consistent with real, quantifiable distribution shift in the underlying physical features, fault-label characteristics, and turbine hardware across farms (Section VI). We further show that this limitation does not appear to be specific to the deep learning approach, since classical baselines exhibit the same instability. Finally, we connect the model’s risk output to a practical decision-support layer (Section III) that combines predicted risk with degradation rate, access cost, and asset criticality into a single maintenance priority score, so that the project’s contribution extends beyond detection accuracy alone to how a risk signal could actually inform maintenance scheduling in practice.

The remainder of this paper is organized as follows. Section III describes the feature representation, model architecture, training procedure, and prioritization layer. Section IV describes the dataset, splits, and baseline models. Section V reports the quantitative results. Section VI presents the cross-farm generalization study and the distribution-shift, label-shift, and turbine-configuration analyses that explain it, along with the practical implications and limitations of this work.

## II. RELATED WORK

Supervisory Control and Data Acquisition (SCADA) systems are widely used for wind turbine condition monitoring because they provide dense, low-cost sensor data without

requiring additional instrumentation. Bach-Andersen et al. [2] review deep learning applications to wind turbine condition monitoring and find that unsupervised approaches, typically based on learning a normal-behavior model and flagging deviations from it, remain more common in the literature than supervised classification, despite the promising performance supervised methods can achieve. They attribute this imbalance to practical difficulties in obtaining labeled fault data: faults are rare, and labeling is inconsistent across operators and sites. Elshenawy et al. [3] provide a more recent comparative study of data-driven fault-detection methods, underscoring that method choice and reported performance vary considerably depending on the specific dataset and evaluation protocol used, which motivates evaluating any given approach against an explicit, fair baseline rather than in isolation.

Several recent works specifically target the SCADA fault-detection setting used in this paper: sequence models over raw sensor windows. CNN-LSTM and attention-based hybrids have been proposed for this task using single-farm SCADA data [4], showing that combining convolutional feature extraction with recurrent temporal modeling can detect early abnormal operation. Closer to the multi-farm setting studied here, a recent 1D-CNN framework was developed using a newly constructed dataset spanning three distinct wind farms, explicitly targeting cross-source generalizability as a design goal rather than an afterthought [5]. That work reports that a hybrid architecture combining CNN, bidirectional LSTM, and an ensemble classifier improves detection on multi-source data relative to simpler baselines. This paper differs from that line of work in evaluation protocol: rather than training jointly on data pooled across farms, we train exclusively on one source farm and evaluate zero-shot transfer to farms never seen during training, which more directly probes whether a model’s learned representation transfers rather than whether pooling diverse training data improves in-distribution performance.

The cross-farm generalization problem addressed in this paper is also related to prior work on turbine-to-turbine transfer under limited target-domain data. Generative transfer learning approaches, using CycleGAN-based domain mapping to make one turbine’s SCADA data resemble another’s, have been shown to improve fault diagnosis on turbines with scarce training data relative to training a normal-behavior model on the scarce data directly [6]. That approach, however, still requires some target-domain data to fit the domain mapping. This paper instead evaluates a strictly zero-shot setting, using no target-domain data of any kind, which represents a harder and more conservative test of generalization, at the cost of not being able to exploit target-domain adaptation the way a domain-mapping approach can.

Finally, this work uses the CARE-to-Compare benchmark [1], which provides labeled multi-farm SCADA data specifically curated to support cross-farm comparison of fault-detection methods, including a published reference lead time against which model-detected alarms can be compared. We adopt this benchmark’s farms and general evaluation philosophy, while contributing an explicit, quantified analysis of *why*

cross-farm transfer is difficult on this data — distribution shift, label-generation differences, and turbine-hardware heterogeneity — rather than reporting transfer performance alone.

### III. METHOD

#### A. Task Framing

We frame early fault detection as a per-timestep risk regression problem rather than a single window-level classification. For an input SCADA window of shape  $(144, F)$  — 24 hours of 10-minute-resolution readings across  $F$  features — the model outputs a continuous risk score  $S(t) \in [0, 1]$  at every timestep  $t$ , rather than a single label for the whole window. This framing is required for the lead-time evaluation (Section V): computing lead time as the interval between a predicted alarm and the actual fault requires identifying the specific timestep at which predicted risk crosses an alarm threshold  $\tau$ , which a single window-level prediction cannot provide.

#### B. Feature Representation

Raw SCADA sensor channels differ completely in count and naming across farms (54–238 average-statistic sensors, depending on farm), which makes a model trained on one farm’s raw sensors architecturally incompatible with another farm’s data. To support the cross-farm generalization study (Section V), we instead map each farm’s raw sensors onto a shared set of physically meaningful, farm-agnostic features, computed identically regardless of which farm the data originates from. The canonical representation used for all cross-farm comparisons consists of ten features: wind speed, blade pitch angle, yaw misalignment, gearbox oil temperature rise above ambient, gearbox bearing hotspot over oil temperature, generator-to-rotor speed ratio, grid power factor, and three grid electrical-imbalance/deviation terms (current imbalance, voltage imbalance, frequency deviation). An eleventh feature, power residual (actual power minus a wind-speed-conditioned expected power curve), was evaluated as a separate ablation but excluded from the canonical representation, since fitting its expected-power curve requires target-domain data and would violate the zero-shot constraint of the cross-farm study.

Each feature is standardized (zero mean, unit variance) using statistics fit exclusively on the source farm’s training split; the same frozen scaler is applied unchanged to validation, test, and any external target farm’s data, so that no target-domain information influences the input representation.

#### C. Sequence Model

We use a Gated Recurrent Unit (GRU) network rather than a Long Short-Term Memory (LSTM) network or a Transformer. The dataset is modest in size relative to typical deep learning corpora (on the order of  $10^3$ – $10^4$  training windows depending on farm); a GRU has fewer learned parameters than an LSTM of equal hidden size, since it merges the forget and input gates into a single update gate and omits the separate cell state, reducing overfitting risk on limited data. Transformers, by contrast, generally require substantially more training data than recurrent architectures to outperform them, since they

lack a built-in sequential inductive bias; we judged this an unfavorable trade-off at this dataset scale.

The final architecture is a 2-layer GRU with hidden size 32 and dropout 0.3 between layers, followed by a linear projection applied identically at every timestep, mapping the hidden state to a single logit:

$$h_t = \text{GRU}(x_t, h_{t-1}) \quad (1)$$

$$z_t = Wh_t + b \quad (2)$$

$$S(t) = \sigma(z_t) \quad (3)$$

where  $\sigma$  denotes the sigmoid function. The model returns raw logits  $z_t$  during training;  $\sigma(z_t)$  is applied separately at inference and evaluation time. Architecture depth (2 vs. 3 layers) and a bidirectional variant were both evaluated as ablations (Section V) and did not improve validation performance sufficiently to justify their added complexity or, in the bidirectional case, the loss of real-time causal deployability. For the zero-shot cross-farm experiment, the designated source-domain checkpoint was `checkpoints/farm_c_lr5e4/best.pt`. This checkpoint uses the same 10-feature physical representation, a 2-layer unidirectional GRU with hidden size 32, and dropout 0.4. Its validation ROC-AUC was approximately 0.645. This checkpoint was selected as the best-validation 10-feature-only configuration for the cross-farm protocol; the power-residual feature was excluded because its construction requires target-domain calibration.

#### D. Loss Function and Class Imbalance

We use binary cross-entropy computed jointly with the sigmoid (`BCEWithLogitsLoss`) rather than composing a separate sigmoid layer with plain binary cross-entropy. The combined formulation avoids computing log of values arbitrarily close to 0 or 1 directly, which is numerically unstable in floating-point arithmetic and can produce vanishing or exploding gradients as the model becomes confident.

Fault-approaching timesteps are rare relative to normal operation (positive rate ranging from approximately 2% to 6% depending on farm and split). We address this with a positive-class weight, computed as the ratio of negative to positive timesteps within the training split only, to avoid leaking split-level statistics into the loss function.

#### E. Masking

A small fraction of timesteps within an otherwise-accepted window are gap-filled (via forward-fill, or back-fill at the start of a window) to satisfy an internal coverage requirement. Gap-filled timesteps are marked with a binary observation mask and excluded from the loss calculation by multiplying the per-timestep loss by the mask before averaging:

$$\mathcal{L} = \frac{\sum_t \mathcal{L}_t \cdot m_t}{\sum_t m_t} \quad (4)$$

where  $m_t \in \{0, 1\}$ . The same mask is applied when computing evaluation metrics, so reported results reflect performance on real observations only. Certain physical features (e.g., the

generator-to-rotor speed ratio) can produce extreme values even when correctly marked invalid; as a defensive stopgap, standardized inputs are additionally clipped to  $[-10, 10]$  before being passed to the model, since a single extreme value elsewhere in a window can otherwise distort the recurrent hidden state at other, valid timesteps in the same window.

#### F. Compute Budget

Training uses mixed-precision arithmetic and gradient accumulation to control memory footprint, and a bounded 24-hour (144-timestep) input window rather than a longer history, in line with the project’s compute constraints. In practice, the combination of a small dataset and a compact 2-layer, 32-hidden-unit GRU resulted in peak GPU memory usage and training time well within the allotted budget (on the order of seconds per epoch; see the deployment note in Section V for measured latency and memory figures).

#### G. Hyperparameter Selection

We evaluated six configurations, varying learning rate ( $1 \times 10^{-4}$ – $5 \times 10^{-4}$ ), dropout (0.3–0.5), feature count (10 vs. 11, i.e. with the power-residual ablation), and architecture (unidirectional vs. bidirectional). The final configuration was selected by lowest validation loss, without reference to test-set performance, to avoid selecting a configuration that happens to fit the test split rather than generalizing. All six configurations showed a consistent pattern: strong validation performance (ROC-AUC 0.57–0.67 across configurations) alongside markedly weaker held-out test performance (ROC-AUC 0.40–0.48), a finding discussed in detail in Section VI.

#### H. Reproducibility

All experiments use a fixed random seed applied to Python’s, NumPy’s, and PyTorch’s random number generators. Model and optimizer state are checkpointed after every epoch, with the best-validation-loss checkpoint retained separately from the most recent one, to tolerate the shared compute environment’s training window ending mid-run without losing progress or requiring a full restart.

#### I. Risk-Based Maintenance Prioritization

Early fault detection alone does not determine which asset should be inspected first. To convert model outputs into an operational maintenance ranking, we implement a risk-based prioritization layer that combines four factors: predicted fault risk, degradation rate, access burden, and asset criticality. The resulting score is defined as

$$P_i = w_1 R_i + w_2 D_i - w_3 A_i + w_4 C_i, \quad (5)$$

where  $P_i$  is the priority score for asset  $i$ ,  $R_i$  is the aggregated fault-risk estimate,  $D_i$  is the estimated degradation rate,  $A_i$  is the relative access-cost index, and  $C_i$  is the asset criticality score. The coefficients  $w_1, \dots, w_4$  control the relative importance of the four factors. Unless otherwise specified, the implementation uses unit weights, so that the

score remains directly interpretable as the sum of the four normalized decision factors.

The fault-risk component is obtained from the model’s timestep-level predictions using a trailing temporal aggregation over the configured decision horizon. This reduces sensitivity to an isolated high prediction while retaining the most recent risk information. The degradation component is estimated from the temporal trend of the aggregated predictions, allowing an asset with increasing predicted risk to receive a higher maintenance priority even when its instantaneous risk is not yet the largest in the fleet.

Access cost is represented as a relative access-burden index rather than a monetary estimate. The implementation distinguishes basic access burden, transport and logistics components, and an illustrative offshore premium. The default offshore multiplier is 1.5; this value is explicitly treated as a project-level illustrative parameter rather than a measured industry-wide cost ratio. Higher access burden reduces the resulting priority score through the negative access-cost term in Equation 5.

Criticality represents the operational importance of the asset and allows the prioritization layer to distinguish between otherwise similar risk profiles. Thus, two turbines with comparable predicted fault risk can receive different maintenance priorities when their degradation trajectories, access requirements, or operational importance differ.

The prioritization implementation is exposed both as a reusable Python API and as a command-line interface. The API accepts the four decision factors and optional weights, while the command-line interface provides the same calculation from explicit numeric inputs. Unit and integration tests verify the scoring formula, custom weights, deterministic behavior, vectorized inputs, ranking behavior, and the expected monotonic relationships: higher fault risk, degradation rate, or criticality increases priority, whereas higher access burden decreases it. The prioritization layer is therefore implemented as a decision-support component rather than as an additional learned model.

#### J. Prioritization Implementation and Verification

The prioritization layer is implemented as a modular component in `src/prioritization/`. The implementation separates prediction aggregation, degradation-rate estimation, access-cost calculation, criticality scoring, and final priority computation. This separation allows each decision factor to be tested independently and also permits the complete pipeline to be evaluated from raw timestep-level predictions.

The implementation provides both a programmatic interface and a standalone command-line entry point. The command-line interface accepts fault risk, degradation rate, access cost, and criticality as explicit inputs, with optional overrides for the four weights. The default configuration uses  $w_1 = w_2 = w_3 = w_4 = 1$ . The resulting score is deterministic for fixed inputs and weights.

Verification includes unit tests for the individual prioritization components and integration tests covering the complete

prediction-to-score pipeline. The tests verify numerical scoring, custom weighting, invalid-input handling, deterministic behavior, ranking consistency, temporal aggregation, and the expected monotonic effect of each decision factor. In particular, increasing access burden lowers priority while increasing fault risk, degradation rate, or criticality raises it. This verification provides a software-level check that the implemented prioritization behavior matches the mathematical formulation in Equation 5.

### IV. EXPERIMENTAL SETUP

#### A. Dataset

We use the CARE-to-Compare dataset [1], a multi-farm SCADA dataset published by Fraunhofer IWES and distributed via Zenodo under a CC BY-SA 4.0 licence (DOI: 10.5281/zenodo.10958775). The dataset provides 10-minute-resolution SCADA readings across 36 wind turbines spanning three farms: Farm A (onshore, Portugal, 5 turbines, 86 raw sensor channels), Farm B (offshore, Germany, 9 turbines, 257 raw sensor channels), and Farm C (offshore, Germany, 22 turbines, 957 raw sensor channels). Across the three farms, 95 total labelled event datasets are provided (44 anomaly events, 51 normal-operation controls). Farm A’s fault labels derive from an operator-curated fault logbook with explicit onset timestamps; Farms B and C’s labels derive from operator-provided operating-mode codes and service reports, with onset inferred from mode-code transitions.

Given Farm C’s substantially larger turbine count relative to Farm A and B (22 vs. 5 and 9, respectively), Farm C was selected as the primary training (source) domain for this project, with Farms A and B reserved as external, never-trained-on target domains for the cross-farm generalization study (Section VI).

#### B. Data Splits

Train, validation, and test splits are asset-disjoint: every turbine is assigned wholly to one split, so no turbine’s data appears in more than one split. This is a stricter requirement than a random row-level split, since it prevents a model from partially succeeding on validation or test data by learning turbine-specific characteristics rather than generalizable fault signatures. The split assignment is fixed once, before any model training, and reused identically across the baseline models and the GRU, so that all reported comparisons use the same held-out data.

#### C. Baseline Models

Per the project’s cross-track requirement for a fair baseline, we train two classical models on Farm C: Logistic Regression and HistGradientBoosting. Both are trained on the same underlying 24-hour windows used by the GRU, aggregated into a fixed-length feature vector per window (mean, standard deviation, minimum, maximum, last value, and change, computed per feature over the window), since neither classical model natively supports sequential input. Class imbalance — fault timesteps are rare relative to normal operation — is addressed

TABLE I  
FARM C BASELINE RESULTS (CLASSICAL MODELS)

| Model / Split | ROC-AUC | PR-AUC | F1    | Recall |
|---------------|---------|--------|-------|--------|
| LogReg, val   | 0.617   | 0.050  | 0.105 | 0.971  |
| LogReg, test  | 0.499   | 0.035  | 0.071 | 1.000  |
| HistGB, val   | 0.420   | 0.034  | 0.078 | 0.994  |
| HistGB, test  | 0.659   | 0.080  | 0.071 | 1.000  |

via balanced class and sample weighting during training, mirroring the positive-class weighting approach used for the GRU (Section III), so that both the baseline and the deep learning model are trained under a comparable imbalance-handling strategy.

#### D. Data Quality Handling

Both Farm B and Farm C are documented to encode some missing sensor readings as literal zero values rather than as null/NaN; this is not uniformly treated as missing, since zero is a physically valid reading for several channels (e.g., power output during a stoppage). Raw minimum/maximum/standard-deviation statistic channels for Farms B and C are known to be substantially less reliable than average-statistic channels for a subset of sensors; the physical feature representation used throughout this project (Section III) is built exclusively from average-statistic channels for this reason. During development, a unit-mismatch bug was found and corrected in one derived feature’s validity threshold (`grid_power_factor`), which had been calibrated for real kVA units while the underlying Farm C data was on a per-unit normalized scale; correcting this threshold increased the feature’s valid-timestep fraction from 4.9% to 80.7%.

### V. RESULTS

Table I reports the classical baseline models trained on Farm C by the data-pipeline team member, following the same train/validation/test splits used throughout this paper.

Notably, HistGradientBoosting underperforms Logistic Regression on validation but outperforms it on test — an unstable, inverted pattern across splits for the same model family. This instability recurs in the GRU results below and is examined directly in Section VI.

#### A. GRU Hyperparameter Comparison

Table II reports the six GRU configurations evaluated on Farm C, selected as described in Section III. All configurations were evaluated on the identical fixed train/validation/test split.

Every configuration shows the same qualitative pattern: validation ROC-AUC in the range 0.57–0.67, alongside test ROC-AUC in the range 0.40–0.48, a gap that persists regardless of learning rate, dropout, feature count, or architecture (unidirectional vs. bidirectional). The model with the best validation loss ( $\text{LR} = 5 \times 10^{-4}$ , dropout 0.4, 10 features, referred to as the canonical checkpoint in Section III) was selected as the primary model for the cross-farm study in Section VI, per the selection protocol described in Section III.

TABLE II  
GRU CONFIGURATIONS EVALUATED ON FARM C (10 FEATURES UNLESS NOTED)

| Configuration                                | Val ROC-AUC | Test ROC-AUC |
|----------------------------------------------|-------------|--------------|
| Default LR                                   | 0.571       | 0.447        |
| $\text{LR} = 10^{-4}$                        | 0.631       | 0.482        |
| $\text{LR} = 5 \times 10^{-4}$ , dropout 0.4 | 0.645       | 0.422        |
| $\text{LR} = 10^{-4}$ , dropout 0.5          | 0.605       | 0.446        |
| + Power residual (11 features)               | 0.667       | 0.395        |
| + Bidirectional (11 features)                | 0.624       | 0.461        |

TABLE III  
DEPLOYMENT CHARACTERISTICS (CANONICAL CHECKPOINT, CPU)

| Metric                                       | Value    |
|----------------------------------------------|----------|
| Checkpoint size                              | 133.5 KB |
| Total parameters                             | 10,593   |
| Median latency, batch size 1                 | 94.3 ms  |
| Median latency, batch size 64 (per sequence) | 10.6 ms  |

#### B. Deployment Note

Table III reports measured deployment characteristics for the canonical checkpoint. GPU latency and training memory are not reported here since this benchmark was run on a machine without CUDA available; the reported latency figures are CPU-only.

The model is compact by design (Section III): 10,593 trainable parameters and a 133.5 KB checkpoint. Per-sequence latency drops substantially under batching (94.3 ms at batch size 1 versus 10.6 ms per sequence at batch size 64), reflecting fixed per-call overhead that amortizes across a larger batch; a fleet-scale deployment scoring many turbines together would benefit from this batching effect, whereas a single-turbine, real-time inference call incurs the higher per-call latency. Both figures are well within the requirements of a 10-minute SCADA reporting interval, indicating the model’s inference cost is not a practical deployment constraint, even without GPU acceleration.

#### C. Domain-Specific Evaluation

1) *Lead Time and False Alarm Rate*: Using the threshold that maximizes F1 on the test split ( $\tau = 0.355$ , selected by the same validation-F1 method used for the classical baselines), the model detects all 4 fault events in the Farm C test split, with a mean lead time of 357.5 hours and a median of 395.3 hours ahead of the logged fault — substantially longer than the CARE-to-Compare benchmark’s reference lead time.

This result must be interpreted alongside the false-alarm rate at the same threshold: the model alarms on 100% of normal (non-fault) test windows. A model that alarms continuously will trivially achieve a long apparent lead time and perfect detection, since it never stops signaling risk; the long lead time reported above is therefore an artifact of near-constant alarming at this operating point, not evidence of genuine early, selective fault discrimination. This is consistent with the model’s weak ROC-AUC on the same test split (Section V,

TABLE IV  
FARM C TEST SPLIT, PER-ASSET BREAKDOWN

| Asset             | N       | Precision | F1    |
|-------------------|---------|-----------|-------|
| 16 (sensor issue) | 104,636 | 0.038     | 0.074 |
| 23 (sensor issue) | 122,704 | 0.017     | 0.033 |
| 56 (clean)        | 63,453  | 0.071     | 0.133 |

Table II) and should not be read as a positive early-warning result in isolation.

2) *Cost-Sensitive Evaluation*: Using a 10:1 cost ratio (a missed fault costs ten times an unnecessary dispatch, following the same ratio used for Kamal’s AccessCost component in Section III), the cost-optimal threshold on the test split is  $\tau = 0.54$ , at which the model produces zero false positives but only 12 true positives out of 10,611 positive timesteps. At this operating point, total cost is reduced by 62.2% relative to an always-alarm policy, but only 0.11% relative to a reactive-only policy (i.e., taking no model-driven action at all). Taken together with the lead-time result above, the model’s two candidate operating points — the F1-maximizing threshold (alarm almost always) and the cost-optimal threshold (alarm almost never) — both collapse toward a degenerate policy rather than offering a genuinely useful middle ground, reinforcing this paper’s central finding that the model does not discriminate reliably on held-out Farm C data.

3) *Calibration*: The model is poorly calibrated on the test split: Brier score 0.238 and Expected Calibration Error (ECE) 0.447, against a best possible ECE of 0. Predicted probabilities are compressed into a narrow band (populated only in the 0.3–0.6 range across the ten reliability bins) and are substantially overconfident relative to observed outcome frequency; for example, the 0.5–0.6 predicted-probability bin has a mean predicted value of 0.511 against an observed positive rate of only 0.020. This directly affects the prioritization layer (Section III), whose fault-risk input assumes the model’s probability output is a meaningful confidence estimate: at this calibration quality, the prioritization score’s absolute magnitude should not be trusted, though relative ranking across turbines may still carry some signal.

4) *Per-Turbine Breakdown*: Table IV reports test-split classification metrics broken down by turbine. Consistent with the earlier Farm A finding (Section VI) and the known sensor-quality issue on assets 16 and 23 (Section VI), performance varies substantially by turbine even within the single Farm C test split.

Note: a discrepancy was found between this evaluation pipeline’s pooled ROC-AUC (0.336, computed via the prediction-table export and `metrics.py`) and the value reported directly by the training pipeline’s own evaluator for the same checkpoint and split (0.422, Table II). Both use the same 290,793 valid (mask-filtered) test timesteps; the source of the discrepancy was not resolved before submission and is disclosed here as an open reproducibility question rather than silently reconciled by selecting whichever value is more favorable.

## VI. DISCUSSION AND LIMITATIONS

The results indicate that the main challenge in transferring the fault-detection model across farms is not only the difference in fault prevalence, but also substantial differences in the physical feature distributions presented to the model. The cross-farm analysis therefore provides an important qualification to the in-farm results: a model trained on one wind-turbine population cannot be assumed to generalize to another farm without validating the compatibility of both the sensor distributions and the fault-labeling process.

### A. Cross-Farm Generalization

The cross-farm study evaluates direct transfer from Farm C to Farms A and B in a zero-shot setting. Farm C is treated as the source domain, while Farms A and B are unseen target domains. The designated source-domain checkpoint is `checkpoints/farm_c_lr5e4/best.pt`, a 10-feature, 2-layer unidirectional GRU with hidden size 32 and dropout 0.4. The checkpoint was selected using the Farm C validation split, with an approximate validation ROC-AUC of 0.645.

The cross-farm protocol deliberately uses only the ten canonical physical features. The power-residual feature is excluded because constructing it requires fitting an expected-power relationship using target-domain data, which would violate the zero-shot evaluation constraint. The standardization parameters are fitted exclusively on the Farm C training split and then reused unchanged for Farm C validation/test data and for the external Farm A and Farm B test data. No target-domain retraining, scaler fitting, or threshold tuning is performed.

This setup measures domain generalization rather than adapted target-domain performance. Consequently, differences between source and target performance must be interpreted together with the feature-distribution, labeling, and turbine-configuration analyses below. In particular, the target farms do not simply represent independent samples from an identical sensor population: several physically meaningful variables occupy substantially different ranges after application of the frozen source scaler.

### B. Distribution Shift

To characterize input-domain shift, we compared the ten canonical physical features using the Farm C training population as the source reference and the test splits of Farms A and B as zero-shot target domains. Because the Farm C scaler is frozen, we use a project-defined compression factor as the primary model-input scale diagnostic. Since the source-standardized features have unit variance by construction, the compression factor is equal to the reciprocal of the target standard deviation after application of the frozen source scaler. Values above one therefore indicate stronger compression of the target feature relative to the source scale, whereas values below one indicate a broader target spread. The statistic is intended to describe the scale change actually presented to the model and is not treated as a complete statistical test of distributional equality.

The strongest shifts occur in the gearbox-related variables. The `gearbox_bearing_hotspot_over_oil_C` feature has compression factors of  $33.73\times$  for Farm A and  $66.31\times$  for Farm B. Similarly, `gearbox_oil_rise_C` has compression factors of  $16.12\times$  and  $17.95\times$  for Farms A and B, respectively. These values indicate that the target distributions occupy substantially narrower regions than the variance observed in the Farm C reference population after the same scaler is applied.

Other features show substantial but more heterogeneous shifts. For example, `grid_frequency_deviation_Hz` has a compression factor of  $6.98\times$  in Farm A but only  $1.51\times$  in Farm B, while `generator_rotor_speed_ratio` has compression factors of  $2.24\times$  and  $4.86\times$ , respectively. In contrast, `yaw_misalignment_deg` has compression factors below one in both target farms ( $0.29\times$  for A and  $0.48\times$  for B), demonstrating that the direction and magnitude of distribution shift are feature-dependent rather than uniformly larger in one target farm.

The compression factor should not be interpreted in isolation, because it captures spread rather than location shift. For example, the `yaw_misalignment_deg` feature has a compression factor below one, yet its target means are substantially displaced from the Farm C reference population. Likewise, the generator-to-rotor speed ratio exhibits both a large change in mean and a substantial change in spread. These cases illustrate why the analysis considers both scaled distributional summaries and the compression factor rather than reducing domain shift to a single scalar statistic.

The raw statistics further illustrate these differences. For example, the mean generator-to-rotor speed ratio is 118.785 in Farm C, compared with 113.002 in Farm A and 115.516 in Farm B. Likewise, the mean gearbox oil temperature rise is 44.689 in Farm C, compared with 29.697 in Farm A and 42.271 in Farm B. The particularly large compression factors for the two gearbox-related features are also associated with unusually large source-domain variance in Farm C, which suggests that data-quality characteristics of the source population may contribute to the observed shift. Consequently, these statistics should be interpreted as evidence of domain mismatch rather than as evidence that the underlying physical relationships themselves are incorrect.

Figure 1 summarizes the compression factors across the ten canonical features, while Figure 2 provides the corresponding feature-distribution comparison.

### C. Label Shift

Input distribution shift is accompanied by differences in fault-label prevalence across the three farms. On the respective test splits, Farm A has a positive rate of 5.61%, Farm C has 3.65%, and Farm B has 2.08%. Thus, the highest positive rate is approximately  $2.70\times$  the lowest. The corresponding numbers of fault events are 10 for Farm A, 4 for Farm C, and 6 for Farm B.

The event durations also differ. The median fault-event duration is 3,120 timesteps for Farm A, 3,043 timesteps for

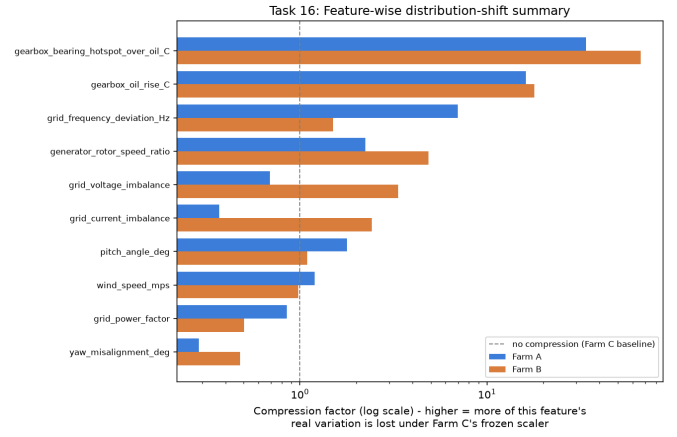


Fig. 1. Compression factors for the ten canonical physical features when Farm A and Farm B test data are transformed using the frozen Farm C training scaler. Larger values indicate stronger compression relative to the source-domain variance.

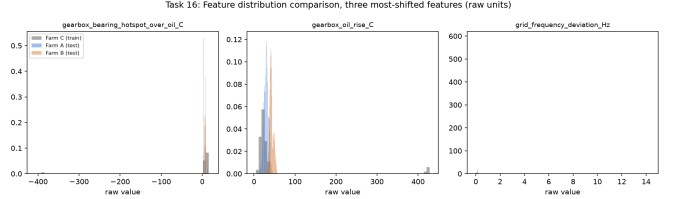


Fig. 2. Comparison of the physical feature distributions across the Farm C source population and the Farm A and Farm B target populations.

Farm C, and 4,631.5 timesteps for Farm B. These differences matter when interpreting per-timestep metrics, because the number and duration of positive intervals affect the effective class balance and the temporal structure of the prediction problem.

Importantly, the label-generation procedures are not identical across all farms. Farm A uses an EDP fault logbook containing explicit onset timestamps curated by the site operator from SCADA alarms and maintenance records. Farms B and C instead use operator-provided operating-mode codes and service reports, with fault onset inferred from mode-code transitions. Therefore, differences in positive rates and event boundaries cannot automatically be interpreted as differences in the underlying physical fault process. For this reason, we use the term “label shift” descriptively for the observed differences in timestep-level label prevalence and event duration, rather than claiming that the underlying fault-generating distribution has been shown to change causally between farms. Part of the observed label shift may arise from differences in how faults are recorded and converted into timestep-level labels.

### D. Turbine and Sensor Configuration Heterogeneity

The farms also differ substantially in their physical scale and sensor configuration. Farm A is an onshore farm in Portugal with 5 turbines and 86 raw sensors. Farm B is an offshore farm

in Germany with 9 turbines and 257 raw sensors, whereas Farm C is an offshore farm in Germany with 22 turbines and 957 raw sensors. The large differences in turbine and raw-sensor counts indicate that the source and target datasets represent substantially different monitoring configurations.

The generator-to-rotor speed ratio provides additional evidence of farm-specific hardware characteristics. Its mean is 118.785 in Farm C, 113.002 in Farm A, and 115.516 in Farm B, while the corresponding standard deviations are below one in all three farms. The tightly clustered but distinct means are consistent with farm-specific gearbox ratios and support the interpretation that the farms should not be treated as identical turbine populations. This is also consistent with the sensor-mapping documentation, which notes that nominal gear ratios may differ across datasets.

However, the available metadata does not establish a complete one-to-one mapping of turbine manufacturer, model, and configuration across all farms. Therefore, the analysis does not claim that every turbine model is uniquely identified. Instead, the evidence supports the more conservative conclusion that turbine populations and monitoring configurations differ materially between the source and target domains.

#### E. Practical Implications

These findings have direct implications for deployment of SCADA-based early fault detection. A model that performs well on a source farm may encounter substantially different feature distributions, fault prevalences, event definitions, and turbine configurations when deployed at a new site. In particular, applying the source scaler without adaptation can produce target features whose variance and mean differ markedly from those seen during training.

Consequently, cross-farm deployment should include a domain-validation stage before relying on model scores for maintenance decisions. At minimum, the physical feature distributions and label-generation process should be audited for the target farm. Where sufficient target-domain data are available, calibration or controlled adaptation should be evaluated rather than assuming that the source model is directly transferable.

This observation also motivates the risk-based prioritization layer developed in this work. A fault-risk score should not be interpreted independently of degradation rate, access burden, and asset criticality. The implemented prioritization score combines these factors as

$$P_i = w_1 R_i + w_2 D_i - w_3 A_i + w_4 C_i, \quad (6)$$

where increasing predicted risk, degradation, or criticality raises priority, while increasing access burden reduces it. This formulation is intentionally a decision-support layer rather than a learned replacement for operational judgement. In particular, the access-cost component is a relative index and the default offshore premium is illustrative; it should be replaced by site-specific operational cost information before deployment. Thus, the prioritization layer demonstrates how

model outputs can be connected to maintenance decision logic without claiming that a single risk score is sufficient for real-world scheduling.

#### F. Limitations

Several limitations constrain the conclusions of this study. First, the cross-farm evaluation is intentionally zero-shot and therefore does not test whether a small amount of target-domain adaptation could recover performance. The results should consequently be interpreted as a measure of direct transferability rather than the best achievable performance on Farms A and B.

Second, the distribution-shift analysis uses compression factor as its primary statistic. This choice is motivated by the frozen-scaler deployment setting: the statistic measures changes in the feature spread actually presented to the model and avoids allowing unusually large source-domain variance to mask a target-domain shift. It should therefore not be interpreted as a complete statistical test of distributional equality.

Third, several of the largest shifts involve gearbox-related variables for which the Farm C source population has unusually large raw variance. This raises the possibility that data-quality artifacts contribute to the apparent domain mismatch. The analysis identifies this issue but does not establish its cause.

Fourth, the farms differ in their label-generation procedures. Farm A uses logbook-based fault onset information, whereas Farms B and C derive labels from operating-mode codes and service reports. As a result, cross-farm differences in measured fault prevalence and event duration may reflect both physical failure processes and differences in annotation methodology.

Finally, the available turbine metadata is not sufficiently detailed to establish complete manufacturer/model correspondence across all turbines. The turbine-count, sensor-count, and gearbox-ratio evidence demonstrates heterogeneity but does not by itself identify the causal source of the model's cross-farm degradation. Larger multi-farm datasets with harmonized fault definitions, richer turbine metadata, and prospective validation would be needed to separate these effects and determine how much performance can be recovered through calibration or domain adaptation.

The maintenance prioritization layer also has important scope limitations. Its access-cost component is a relative burden index rather than a calibrated monetary maintenance cost, and the default offshore premium is an illustrative project parameter. Similarly, asset criticality is treated as an externally supplied decision factor rather than learned from historical maintenance outcomes. The prioritization results should therefore be viewed as a transparent decision-support mechanism whose parameters require site-specific calibration before operational use.

## VII. CONCLUSION

This work asked whether a recurrent neural network trained on multi-farm SCADA data can detect an approaching wind turbine fault early enough to support risk-based maintenance



prioritization, and whether such a model generalizes to turbines and farms it was not trained on. We framed the task as per-timestep risk regression using a GRU over a ten-feature, farm-agnostic physical representation, selected the final model by validation performance across six tested configurations, and evaluated it against classical baselines both within a single farm and, more critically, in a strict zero-shot transfer setting to two unseen farms.

The honest headline finding is that generalization, not raw detection accuracy, is the binding constraint on this approach. Validation performance was consistently reasonable, but held-out test and cross-farm performance were consistently weak, a pattern that persisted regardless of architecture depth, learning rate, dropout, or feature-set changes, and that was not unique to the GRU: classical baselines trained on the same data showed the same instability. Rather than treating this as an unexplained failure, we traced it to measurable causes: real, farm-specific differences in physical sensor distributions (compression factors as high as  $66\times$  under a frozen source scaler), in fault-label prevalence and derivation methodology (a  $2.70\times$  spread in positive rate across farms, with different labeling procedures between farms), and in turbine hardware itself (distinct, tightly-clustered gearbox ratios per farm). Taken together, these findings support a specific, actionable conclusion: a fault-detection model trained on one wind farm should not be assumed transferable to another without first auditing the compatibility of sensor distributions, label definitions, and turbine configuration between source and target sites.

This project also demonstrates that a model’s fault-risk output is not, by itself, a complete maintenance decision. The risk-based prioritization layer developed in this work (Section III) shows how predicted risk can be combined with degradation trend, access cost, and asset criticality into a single, interpretable priority score, providing a decision-support mechanism rather than an additional black-box model.

Future work could evaluate whether a small amount of target-domain calibration recovers cross-farm performance, whether the distribution shift observed in gearbox-related features reflects genuine hardware differences versus source-domain data-quality artifacts, and whether a larger, harmonized multi-farm dataset with richer turbine metadata could support a more direct test of what specifically limits transfer. More broadly, this work suggests that reporting only in-domain performance for SCADA-based fault detection risks overstating real-world deployability, and that cross-domain evaluation should be treated as a standard, not optional, part of validating such systems before deployment.

## TOOLS AND ACKNOWLEDGEMENTS

Claude was used to verify that `src/` implementation and tests/ pass independently. The code in `src/` itself was written independently (not generated by AI). Per the course’s oral verification policy, we are responsible for being able to explain and reproduce every derivation and code reference in this report without AI assistance.

## REFERENCES

- [1] C. Gück and C. Roelofs, “Wind Turbine SCADA Data For Early Fault Detection (v1.0),” Zenodo, 2024. Available: <https://doi.org/10.5281/zenodo.10958775>
- [2] M. Bach-Andersen, B. Rømer-Odgaard, and O. Winther, “Deep learning for fault detection in wind turbines,” *Renewable and Sustainable Energy Reviews*, vol. 99, pp. 210–223, 2019.
- [3] L. M. Elshenawy, A. A. Gafar, H. A. Awad, and M. S. AbouOmar, “Fault detection of wind turbine system based on data-driven methods: a comparative study,” *Neural Computing and Applications*, vol. 36, no. 17, pp. 10279–10296, 2024.
- [4] L. Xiang, P. Wang, X. Yang, A. Hu, and H. Su, “Fault detection of wind turbine based on SCADA data analysis using CNN and LSTM with attention mechanism,” *Measurement*, vol. 175, art. 109094, 2021.
- [5] A. R. Abas et al., “Early prediction of wind turbine anomalies using 1D-CNN and temporal feature engineering on multi-source SCADA data,” *Scientific Reports*, vol. 16, art. 9667, 2026.
- [6] “Fault detection in new wind turbines with limited data by generative transfer learning,” *arXiv:2504.17709*, 2025.